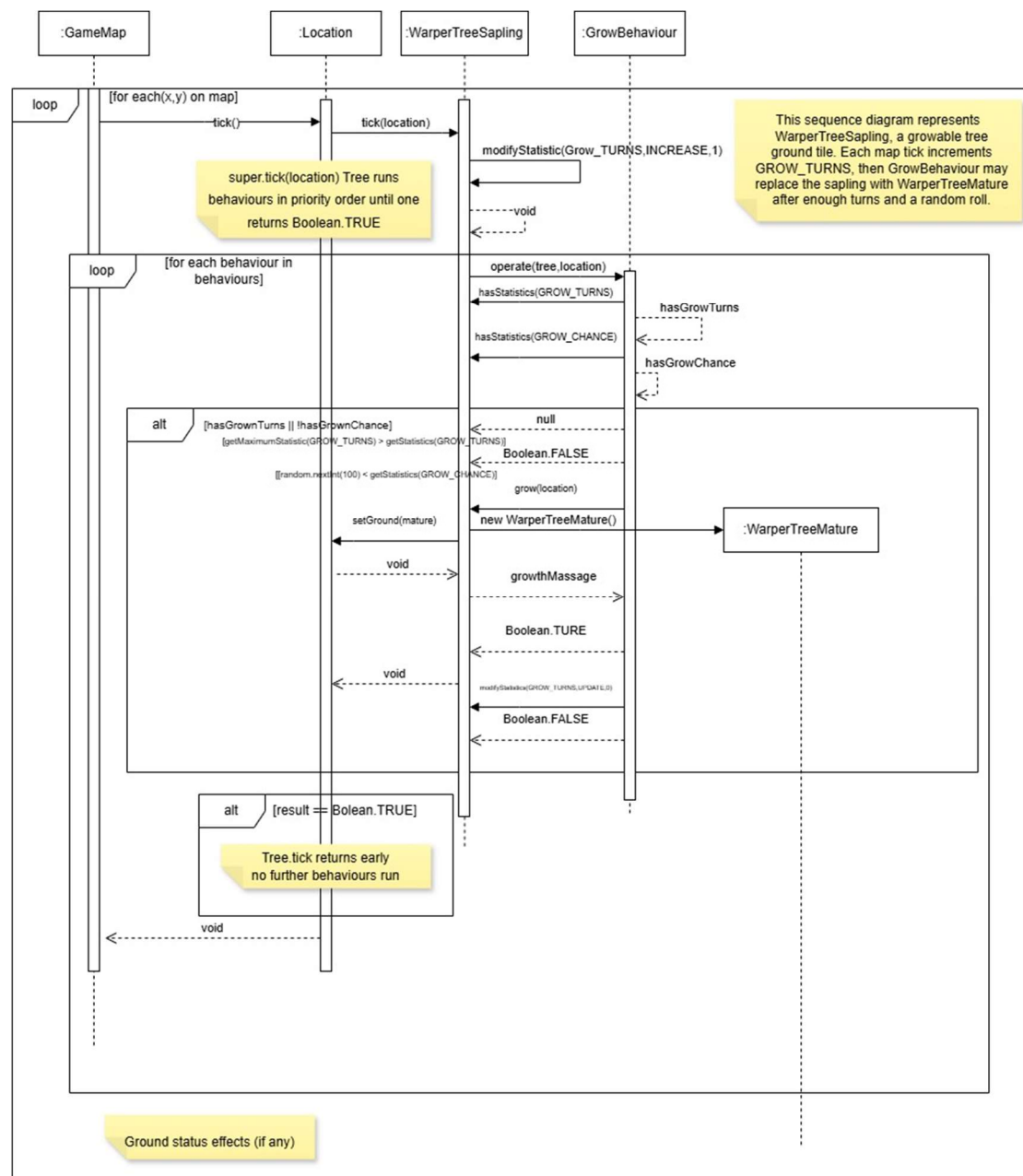


edu.monash.it2016.engine

Sequence Diagram



REQ 3 Design Rationale

Design 1: Prioritized Behaviour Pipeline

Abstract Tree stores behaviours in a TreeMap keyed by priority. FleshyTreeSprout registers GrowBehaviour then SpawnBehaviour; WarperTreeMature registers TreeTeleportBehaviour. Growable stages increment GROW_TURNS before Tree.tick, which runs behaviours until one returns TRUE. One-action-per-turn rule follows because successful growth ends the tick before spawning. KISS uses one scheduler; DRY avoids duplicating turn control. SRP splits scheduling from effects; OCP adds flora via new behaviours. Example: on turn 20 a nearby worker triggers spawn only if the 25% growth roll fails. Maintenance stays stable because warp and spawn rules do not interleave in one method.

Connascence of position (priority keys) and execution (first TRUE stops the chain) are moderate: coupling is localized to registration order. Extension: ThornSnareBehaviour depends on Tree and registers on a new ThornSprout.

Pros	Cons
Single loop enforces one flora action per turn.	Priority values are fragile implicit contracts.
Effects compose without editing Tree.	Return-value conventions must stay uniform.
Warp, grow, and spawn stay in separate units.	Turn outcomes need tracing across behaviours.

Design 1: Alternative Design

Each stage would override tick with inline growth, spawn, and warp checks, returning after the first success. Growth timers and adjacency scans repeat across classes, weakening DRY. KISS looks simpler locally; SRP suffers when one method owns randomness, actors, and map updates. Example: sprout logic would duplicate worker detection now in SpawnBehaviour. Connascence of algorithm is stronger than the pipeline because parallel copies must stay aligned. Trade-off: fewer types, more regression risk when tuning priorities. Extension: ParasiticTree adds another large tick instead of a behaviour.

Pros	Cons
All logic visible in one method per stage.	Single-action ordering duplicated per class.
No behaviour map to document.	Spawn and grow code diverge over time.
	Harder to reuse warp logic on warper mature.

Design 2: Growable Ground Stages

Growable.grow replaces the tile through location.setGround, giving each stage its own class (FleshyTreeSprout, FleshyTreeSapling, WarperTreeSapling, etc.). GrowBehaviour reads TreeStatistics for 20/25-turn windows and 25%/50% chances, then delegates replacement. Mature fleshy trees appear only when sapling growth constructs FleshyTreeMature with spawners. Encapsulation keeps glyphs and timers per stage. DRY shares timed growth; ISP keeps non-growing mature trees off Growable. Example: after 25 sapling turns, a failed 50% roll resets the counter; success becomes Y. Scalability adds species by chaining new grounds. Connascence of type (GrowBehaviour needs Growable) and meaning (enum stats encode growth rules) are moderate. Extension: SporeBud implements Growable and yields SporeCanopy, reusing GrowBehaviour.

Pros	Cons
Map display always matches the active stage class.	Each stage needs a new class and grow link.
One growth algorithm serves all evolving flora.	Stat names must match GrowBehaviour expectations.
Stage-specific stats stay encapsulated.	Class count grows with every species line.

Design 2: Alternative Design

One FleshyTree would use a Stage enum, mutating character, timers, and spawn flags internally. Fewer files aid KISS superficially, but warper and fleshy rules converge in one class, hurting SRP and OCP. Example: maturity would flip enum rather than call setGround. Connascence of

meaning intensifies inside the enum-driven class, weaker between files but harder to change one species. Trade-off: less allocation, denser conditionals. Extension: WARPED adds more switches, increasing coupling.

Pros	Cons
Fewer types for the fleshy chain.	Fleshy spawn and warper warp share one switch tree.
Transitions avoid new ground instances.	Enum edits risk breaking unrelated stages.
Shared fields can persist across stages.	Harder to read stage behaviour from the type alone.

Design 3: Injected Spawner Strategy

SpawnBehaviour takes List<Spawner>; Overflow20 passes SlimeSpawner to sprouts, while FleshyTreeSapling.grow attaches UndeadSpawner to mature trees. spawnAt creates actors and runs slime drop or undead HP reactions. DIP keeps trees on Spawner; OCP adds creatures via new spawner classes. Example: worker beside Y picks a neighbour tile and UndeadSpawner.spawnAt runs. Connascence of name (TREE_ACTIVATOR) and type (list element contract) stay mild. Extension: FleshyTreeMature(List.of(new SlimeSpawner(), new UndeadSpawner())) mixes hazards without changing SpawnBehaviour.

Pros	Cons
Spawn consequences live with the creature type.	Misconfigured lists fail at runtime.
Maps configure pools per tree instance.	Worker detection remains in SpawnBehaviour.
Trees avoid duplicating undead or slime effects.	Random pool choice hides outcomes until spawn.

Design 3: Alternative Design

Trees would construct Slime or Undead directly when workers are adjacent. Undead boosting and item drops duplicate wherever spawning occurs, breaking DRY. Connascence of type to concrete actors is stronger than the interface approach. Trade-off: linear reading, wide change blast radius. Extension: each tree edits its own spawn block for parasites.

Pros	Cons
Direct spawn calls are easy to follow.	Environmental reactions repeat across callers.
No spawner list wiring in maps.	Trees couple to actor constructors.
Minimal abstraction for two creatures.	Mixed pools need new conditionals per tree.

Comparison of Alternative Designs

Fat tick overrides maximize algorithm connascence across stages. Enum staging reduces inter-class coupling but concentrates meaning connascence in one growing class. Direct spawning minimizes interfaces yet binds trees to concrete actors. The enum alternative weakest spreads coupling across files, but its class becomes the main maintenance hotspot.

CONCLUSION

